

Alocador de Memória Customizado

Isabella Barela Venancio¹, Gabriela Borborema Paes Barreto¹

¹Departamento de Informática – Universidade Estadual de Maringá (UEM)
CEP 87020-900 – Av. Colombo, 5790 - Câmpus Universitário – Maringá – PR – Brasil

{ra144023@uem.br, ra144438@uem.br} sec-din@uem.br

Resumo. *Este trabalho desenvolve um alocador de memória customizado com heap simulado, utilizando as estratégias First-Fit, Best-Fit e Worst-Fit. O projeto também aplica splitting e coalescência, comparando as estratégias por meio de métricas de fragmentação, falhas e desempenho.*

1. Introdução

Este trabalho de Sistemas Operacionais desenvolve a criação de um alocador de memória customizado, operando de forma similar às funções malloc e free. Para isso, será empregado um heap simulado, onde o controle dos blocos de memória será feito manualmente.

O objetivo é aprofundar o entendimento sobre gerenciamento de memória, abordando a organização de blocos livres e ocupados, a fragmentação interna e externa, as diferentes estratégias de alocação e a união de blocos livres. Aliás, posteriormente, faremos uma avaliação experimental de First-Fit, Best-Fit e Worst-Fit, considerando indicadores de fragmentação e o desempenho das operações.

2. Conceitos

2.1. Gerenciamento de memória

A memória principal é um recurso essencial, porém limitado, para sistemas computacionais modernos. Para otimizar o uso da CPU e o tempo de resposta, o sistema operacional mantém múltiplos programas na memória simultaneamente, o que exige mecanismos de gerenciamento adequados às aplicações e ao projeto do hardware. [Silberschatz et al. 2013] e [Machado e Maia 2013].

Entre suas principais responsabilidades estão controlar as regiões em uso, identificar os processos que as utilizam, realizar novas alocações e liberar os espaços quando os processos encerram suas atividades.

O gerenciamento de memória pode ser analisado em diferentes níveis. No nível físico, são gerenciados os endereços da memória RAM; no espaço de núcleo, quadros são destinados a estruturas internas de controle e buffers; e em nível mais alto, as bibliotecas do sistema gerenciam o heap para atender às requisições de alocação e liberação. [Silberschatz et al. 2013], [Machado e Maia 2013] e [Maziero 2020].

2.2. Gerência de heap e blocos livres/ocupados

A gerência de memória dinâmica ocorre por meio do controle do heap do processo. O alocador mantém o registro das regiões livres e ocupadas utilizando estruturas de dados,

permitindo atender às requisições de alocação e liberação sem a intervenção direta do núcleo do sistema operacional.[Maziero 2020] e [Wilson et al. 1995].

Em heaps organizados em blocos, cada segmento possui metadados, geralmente armazenados em um cabeçalho antes da área de dados. Esse cabeçalho contém informações como o tamanho do bloco e seu estado, indicando se está livre ou ocupado.[Jones et al. 2012] e [Wilson et al. 1995].

Nos alocadores customizados, os blocos podem formar uma lista duplamente encadeada, com ponteiros para os blocos anterior e posterior. Ao receber uma requisição, o sistema percorre os metadados em busca de uma região livre compatível. Se o bloco encontrado for maior que o necessário, pode ser realizado o *splitting*. Na liberação, os metadados são atualizados para tornar o espaço novamente disponível.[Machado e Maia 2013].

2.3. Fragmentação interna vs. externa

A fragmentação externa ocorre quando o espaço livre total na memória é disperso em pequenos blocos separados por segmentos ocupados. Apesar de que a soma dessas áreas vazias possa ser suficiente para atender a uma requisição, o alocador não consegue utilizá-las caso nenhum bloco, seja grande o bastante para a demanda. [Jones et al. 2012], [Silberschatz et al. 2018], [Silberschatz et al. 2013] e [Wilson et al. 1995].

A fragmentação interna ocorre pelo desperdício de espaço dentro das partições ou blocos de memória efetivamente alocados. Como a memória física é distribuída em unidades fixas, o espaço excedente dentro do bloco concedido permanece inacessível para outros processos. O tamanho desse desperdício equivale exatamente à diferença entre a memória requisitada e a fisicamente alocada. [Jones et al. 2012], [Silberschatz et al. 2013] e [Silberschatz et al. 2018].

2.4. Estratégias de alocação

First-Fit Varre a lista de blocos livres, alocando o primeiro espaço encontrado que seja grande o suficiente. É geralmente a estratégia mais rápida, mas tende a acumular pequenos fragmentos de memória livre no início da lista, o que pode atrasar novas buscas. [Silberschatz et al. 2018], [Silberschatz et al. 2013], [Maziero 2020], [Stallings 2018], [Feltrim 2026] e [Jones et al. 2012].

Best-Fit Realiza uma varredura por toda a lista de blocos disponíveis, buscando o menor bloco livre capaz de atender à requisição. Porém, o heap tem a possibilidade de se encher de fragmentos pequenos demais para atender a novas requisições, o que aumenta a necessidade de coalescência.[Silberschatz et al. 2018], [Silberschatz et al. 2013], [Stallings 2018], [Feltrim 2026] e [Machado e Maia 2013].

Worst-Fit Costuma apresentar os piores resultados entre as três estratégias, com baixa eficiência temporal e alto desperdício de memória. Ele funciona alocando o maior bloco livre disponível, o que gera uma sobra maior após a divisão, sobra essa que permanece útil para alocações futuras. [Silberschatz et al. 2018], [Silberschatz et al. 2013], [Feltrim 2026], [Stallings 2018] e [Machado e Maia 2013].

2.5. Splitting e coalescência/merging

Quando o alocador seleciona um bloco livre que é maior do que o tamanho requisitado pela aplicação, ele executa a operação de splitting. O segmento original é particionado, a primeira é destinada a atender à solicitação do programa e a sobra restante é registrada como um novo e menor bloco disponível no heap, reduzindo o desperdício de memória. [Jones et al. 2012] e [Wilson et al. 1995].

As várias operações de liberação de memória costumam gerar fragmentos espalhados que não conseguem realizar novas requisições. Para solucionar, o gerenciador aplica a coalescência ao liberar espaço, consiste em analisar se os blocos adjacentes na memória também estão desocupados para mesclar em uma única área maior. Consolidar esses fragmentos em blocos maiores, reduzindo a fragmentação externa. [Wilson et al. 1995] e [Silberschatz et al. 2013].

3. Metodologia

3.1. Ambiente de desenvolvimento e experimentação

- Sistema operacional: Linux / Ubuntu 26.04 LTS / WSL;
- Linguagem: C e Python;
- Compilador: GCC;
- Processador: AMD Ryzen 5 7520U with Radeon Graphics;
- Quantidade de memória RAM: 16GB;
- Ferramentas utilizadas: VSCode

3.2. Estrutura do alocador

A estrutura do alocador é baseada em uma lista duplamente encadeada de blocos de memória de tamanhos variados que podem estar disponíveis ou ocupados. Cada bloco de memória é composto por um cabeçalho, que possui informações sobre o tamanho do bloco, se ele está livre ou ocupado e dois ponteiros: um apontando para o próximo bloco e outro apontando para o anterior, e o espaço real de dados úteis.

Na alocação de memória pelo usuário, o alocador retorna o endereço inicial dos dados úteis, logo após os metadados. Em contrapartida, ao receber o ponteiro para a liberação da memória, o alocador subtrai o tamanho do struct de metadados para localizá-los, marcando esse bloco como livre.

3.3. Estrutura dos metadados

```
1 typedef struct bloco {  
2     size_t tamanho;  
3     int livre;  
4     char padding[4];  
5     struct bloco* proximo;  
6     struct bloco* anterior;  
7 } Bloco;
```

- `size_t tamanho`: tamanho ou contagem de bytes do espaço associado ao bloco. Ocupa 8 bytes.

- `int livre`: funciona como uma flag (onde 1 significa livre e 0 significa ocupado) para indicar se o bloco está disponível para alocação. Ocupa 4 bytes.
- `char padding[4]`: vetor de preenchimento, evitando desperdício invisível e otimizando o acesso à memória. Ocupa 4 bytes.
- `bloco* proximo`: ponteiro para o próximo bloco. Ocupa 8 bytes.
- `bloco* anterior`: ponteiro para o bloco anterior. Ocupa 8 bytes.

O tamanho total dos metadados é 32 bytes considerando sistemas de 64 bits, essa informação é necessária ao calcular a memória total a ser pedida ao sistema operacional, ao retornar o ponteiro correto para o usuário ao alocar memória, no momento de liberação de memória a partir do ponteiro enviado pelo usuário e também na divisão de blocos (split) realizado na alocação de memória.

A lista duplamente encadeada é importante para o gerenciamento de blocos de memória pela coalescência, em que o ponteiro anterior permite fundir blocos livres adjacentes em tempo constante $O(1)$, evitando fragmentação, os ponteiros mantêm os blocos encadeados na mesma ordem em que estão dispostos fisicamente no heap. Além disso, ela permite a remoção rápida ao retirar um bloco, alterando apenas os ponteiros dos adjacentes, em tempo constante $O(1)$.

3.4. Inicialização do heap

A inicialização do heap é o ponto de partida do alocador. O programa irá solicitar ao sistema operacional um agrupamento inicial de memória bruta utilizando funções como `sbrk` ou `mmap`, porém para fins didáticos, utilizaremos uma chamada utilizando `malloc` para simular um bloco inicial de memória bruta.

3.5. Processo de alocação

O processo de alocação em uma lista duplamente encadeada ocorre em cinco etapas principais.

Primeiro, é realizar um ajuste de alinhamento, padding, arredondando o tamanho solicitado pelo usuário para o próximo múltiplo de 8 bytes. Por exemplo, ao ser solicitado 5 bytes, seriam alocados 8 bytes, considerando sistemas de 64 bits.

O segundo passo é caminhar pela lista buscando por um bloco livre que atenda duas condições: o bloco está disponível e o bloco é grande o suficiente para alocar a quantidade de memória solicitada. Nessa busca, podem ser utilizadas estratégias como o First-Fit, que seleciona o primeiro bloco adequado, ou a Best-Fit, que procura o bloco com tamanho mais próximo ao solicitado, e o Worst-Fit.

Em seguida, é feita uma avaliação de divisão (o processo de splitting), se o espaço restante é menor do que o tamanho do próprio cabeçalho, não seria possível alocar um novo bloco naquele espaço, então o bloco é retornado inteiro. Caso contrário, o bloco atual é encolhido para o tamanho exato solicitado, enquanto um novo cabeçalho é colocado no restante do espaço disponível e é marcado como livre, atualizando de forma correta os ponteiros do próximo bloco e do bloco anterior.

Por fim, ocorrem as duas últimas etapas: os metadados do bloco que será entregue são alterados para marcar que aquele espaço está ocupado e logo em seguida é retornado o ponteiro apontando para o início dos dados, pulando o cabeçalho para ser utilizado pelo usuário.

3.6. Processo de liberação

O processo de liberação é iniciado quando o usuário chama a função de desalocação, passando como parâmetro apenas o ponteiro para a área de dados úteis. Para acessar os metadados do bloco, o alocador recua na memória o tamanho exato do cabeçalho, em bytes.

Após localizar o cabeçalho, o estado do bloco é alterado para livre e inicia-se o processo de coalescência, no qual os blocos vizinhos são verificados para identificar se também estão livres. Como o alocador utiliza uma lista duplamente encadeada, esse processo é facilitado e pode ser realizado em tempo constante.

Quando há blocos livres adjacentes, eles são mesclados em um único bloco maior, reduzindo a fragmentação externa. No melhor caso, o bloco liberado e seus dois vizinhos estão livres e podem ser unidos em uma única região de memória.

3.7. Tratamento de erros

Para garantir que o usuário não quebre o funcionamento, iremos tratar alguns dos erros mais comuns que podem ocorrer:

- `ERR_HEAP_NOT_INITIALIZED`: Esse erro deve ser tratado quando o usuário tentar chamar a função de alocação ou liberação de memória sem antes ter inicializado o heap.
- `ERR_OUT_OF_MEMORY`: Não há bloco livre grande o suficiente para alocar a memória solicitada
- `ERR_BAD_POINTER`: Acontece quando o usuário passa um ponteiro que não pertence ao heap
- `ERR_DOUBLE_FREE`: Deve ser tratado se o usuário tentar liberar um bloco de memória que já estava livre

3.8. Métricas

As métricas serão utilizadas para comparar as estratégias First-Fit, Best-Fit e Worst-Fit. Os resultados de cada workload e estratégia serão armazenados em `resultados/comparativo.csv`.

Utilizaremos a linguagem de programação Python para análise, com a biblioteca Pandas para leitura e organização dos resultados, e NumPy para cálculo da mediana e percentis.

Fragmentação interna Mede o espaço desperdiçado dentro dos blocos alocados, principalmente devido ao alinhamento. O valor será acumulado durante as alocações, sendo um somatório $\sum$.

`fragmentacao_interna_total += tamanho_real_alocado - tamanho_requisitado`

Ao liberar um bloco (free), seu valor será subtraído, mantendo a métrica em tempo real. Também será calculada a Taxa de Desperdício Interno: $\frac{\text{fragmentacao_interna_total}}{\text{memoria_total_alocada}}$.

Fragmentação externa Mede o espaço livre do heap distribuído entre blocos pequenos. Sempre que uma alocação falhar, será identificado o maior bloco livre e calculada a soma de todos os blocos livres.

Fragmentacao Externa = $1 - \left(\frac{\text{maior_bloco_livre}}{\text{total_free_space}} \right)$ (resultando em um percentual de 0 a 1).

Número de operações por alocação Será utilizado um contador para registrar quantos blocos são analisados durante a busca por uma região adequada. Inserimos uma variável na função de alocação que incrementa a cada nó/bloco da free list inspecionado, então toda vez que o loop `while(bloco_atual)` executa uma checagem `if (bloco_atual->tamanho >= tamanho_pedido)`, faça `ops_contador++`.

Serão registradas as operações totais, as operações por alocação e sua média, permitindo comparar o custo das estratégias First-Fit, Best-Fit e Worst-Fit.

Taxa de falha de alocação Contabilizaremos o total de solicitações de alocação e total de alocações que falharam, com os resultados dados em porcentagem %.

$$\text{taxa_falha_alocacao} = \left(\frac{\text{alocacoes_falhas}}{\text{total_solicitacoes}} \right) \times 100$$

O `total_solicitacoes` será incrementado em toda chamada de alocação. E a `alocacoes_falhas` será incrementado apenas quando a busca por bloco livre retorna NULL e `total_espaco_livre >= tamanho_pedido` (caracterizando falha por fragmentação).

Tempo de execução Vamos medir a latência física (em nanossegundos) para processar cada alocação e liberação. Com a função `clock_gettime(CLOCK_MONOTONIC_RAW)`, vamos obter os resultados e analisar pela Mediana (*p*50) para ignorar interferências do sistema operacional e pelo Percentil 95 (*p*95) para avaliar o desempenho no pior caso.

3.9. Organização do projeto

- `alocador.h`: declarações das estruturas, constantes e funções.
- `alocador.c`: implementação das estratégias do alocador.
- `testes.c`: testes básicos obrigatórios de funcionamento.
- `workload1.c`: alocações sequenciais de tamanhos variados sem liberação — mede uso puro de espaço.
- `workload2.c`: alocações e liberações intercaladas (padrão aleatório) — mede fragmentação.
- `workload3.c`: padrão adversarial: alocar N blocos pequenos, liberar blocos alternados, tentar alocar um bloco grande — testa se a estratégia sofre fragmentação externa severa.
- `comparativo.csv`: armazenamento dos resultados dos experimentos.

3.10. Funções principais

```
1 void mem_init(size_t heap_size);
2 void* meu_malloc(size_t tamanho);
3 void meu_free(void* ptr);
4 void mem_stats(void);
5 void mem_dump(void);
```

- `void mem_init(size_t heap_size)`: Essa função recebe um tamanho, inteiro positivo, que simboliza o tamanho total do heap a ser disponibilizado para o alocador. A função irá utilizar o `malloc` padrão da linguagem C para alocar a quantidade de memória solicitada e irá guardar o ponteiro em uma variável global.

- `void* meu_malloc(size_t tamanho):` Essa função recebe um tamanho, inteiro positivo, que simboliza o tamanho solicitado da memória solicitada pelo usuário e retorna um ponteiro que indica exatamente a posição dos dados úteis no bloco de memória alocado.
- `void meu_free(void* ptr):` Essa função recebe um ponteiro indicando o bloco de memória que o usuário deseja liberar. Ele utilizará o tamanho do struct de metadados para encontrar o início do bloco e liberar a memória utilizando também o processo de coalescência.
- `void mem_stats(void):` Essa função retorna um relatório sobre o uso da memória pelo alocador, por exemplo, informações sobre fragmentação memória ocupada real vs memória solicitada pelo usuário, etc.
- `void mem_dump(void):` Essa função cria uma visualização do estado atual do heap, mostrando em forma de texto os blocos livres e ocupados e o tamanho de cada um, no formato de uma lista duplamente encadeada.

4. Conclusão

Nesta etapa, relatório inicial, definimos a estrutura do alocador, o funcionamento das operações de alocação e liberação e as métricas que serão utilizadas nos experimentos/workloads. Após a execução dos testes, será possível comparar o comportamento das estratégias em relação à fragmentação interna e externa, número de operações, taxa de falha e ao tempo de execução (métrica opcional), permitindo identificar qual estratégia apresenta melhor aproveitamento da memória e melhor desempenho em cada cenário de teste.

Referências

- Feltrim, V. D. (2026). Organização e recuperação de dados. Slides da disciplina Organização e Recuperação de Dados, Departamento de Informática, Universidade Estadual de Maringá. Disponibilizado via Google Classroom.
- Jones, R., Hosking, A., e Moss, E. (2012). *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Press, 1st edition.
- Machado, F. B. e Maia, L. P. (2013). *Arquitetura de Sistemas Operacionais*. LTC, 5th edition.
- Maziero, C. (2020). Sistemas operacionais: Gestão de memória – alocação de memória. Departamento de Informática, Universidade Federal do Paraná.
- Silberschatz, A., Galvin, P. B., e Gagne, G. (2013). *Fundamentos de Sistemas Operacionais*. LTC, 9th edition.
- Silberschatz, A., Galvin, P. B., e Gagne, G. (2018). *Operating System Concepts*. Wiley, 10th edition.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles*. Pearson, 9th edition.
- Wilson, P. R., Johnstone, M. S., Neely, M., e Boles, D. (1995). Dynamic storage allocation: A survey and critical review. In *Proceedings of the International Workshop on Memory Management*, Berlin, Heidelberg. Springer.